

SIMA Architecture Specification

Short-Form Indexed Memory Architecture

Version: 0.1.0

Status: Experimental / Open Research

1. Purpose

This document specifies the architecture of the Short-Form Indexed Memory Architecture (SIMA).

SIMA is a model-agnostic architecture for persistent AI memory that separates:

Storage Representation

from:

Retrieval Representation

The system is designed around progressive retrieval.

Rather than immediately loading complete memory records into an active model context, SIMA attempts to identify relevant memories using compact retrieval representations and only expands full memory records when necessary.

The architecture is intended to investigate whether this separation can improve:

- Retrieval efficiency
- Memory scalability
- Context efficiency
- Long-term recall
- Multi-hop recall
- Retrieval latency

No performance improvement is assumed without empirical validation.

2. Design Principles

SIMA follows six primary principles.

2.1 Separate storage from retrieval

The representation required to preserve a memory is not necessarily the representation required to find it.

Therefore:

Persistent Memory
≠
Retrieval Representation

2.2 Progressive expansion

Retrieval should move from cheap representations to expensive representations.

Category
↓
Index
↓
Short Form
↓
Candidate
↓
Detailed Memory
↓
Full Memory

2.3 Search before expansion

The system should avoid loading full memory objects before determining relevance.

Bad:

Query
↓
Load large memory objects
↓
Determine relevance

SIMA:

```
Query
↓
Search compact representations
↓
Rank
↓
Expand relevant memories
```

2.4 Adaptive retrieval depth

The system should retrieve more information when confidence is low and less information when confidence is high.

```
High confidence
  → shallow retrieval

Low confidence
  → deeper retrieval
```

2.5 Explicit relationships

Memories should be able to reference other memories.

This enables graph-based retrieval.

```
Memory A
|
├── related_to → Memory B
|
├── extends → Memory C
|
└── contradicts → Memory D
```

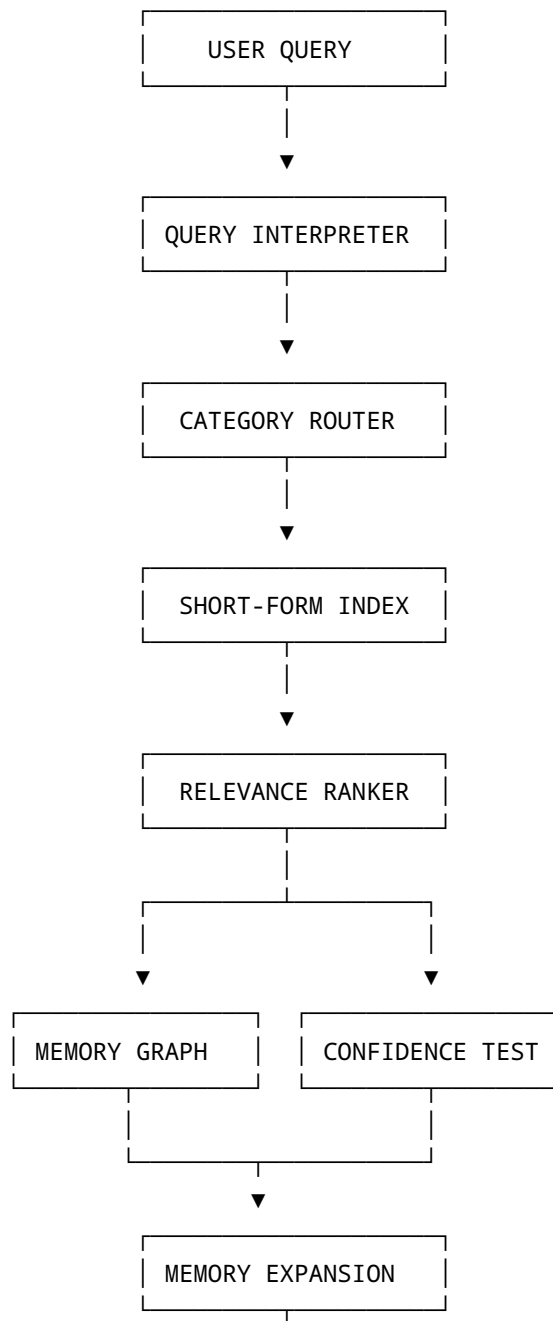
2.6 Reproducible evaluation

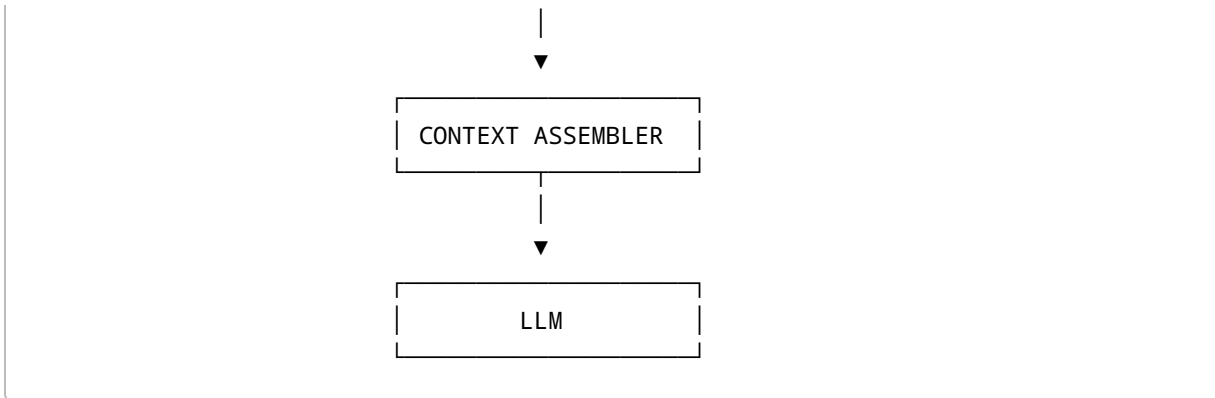
All architectural claims should be validated through measurable benchmarks.

No latency, token-efficiency, or recall improvement should be considered established until reproduced experimentally.

3. System Architecture

The complete system is:





4. Memory Storage Architecture

Each memory exists as a persistent object.

The object consists of:

- Memory ID
- Category
- Subcategory
- Retrieval Key
- Short Form
- Entities
- Topics
- Importance
- Confidence
- Timestamp
- Source
- Full Content
- Embedding Reference
- Relationships

Conceptual representation:

```
{
  "memory_id": "mem_000001",
  "category": "ai_research",
  "subcategory": "memory_architecture",
  "retrieval_key": "SIMA",
  "short_form": "Category-indexed compressed memory retrieval architecture.",
  "entities": ["memory", "retrieval", "context"],
  "topics": ["AI", "architecture"],
```

```
{
  "importance": 0.91,
  "confidence": 0.87,
  "timestamp": "2026-07-21T00:00:00Z",
  "source": "conversation",
  "full_content_ref": "storage://mem_000001",
  "embedding_ref": "vector://mem_000001",
  "relationships": []
}
```

The full content may be stored separately from the retrieval index.

This allows the retrieval layer to remain lightweight.

5. Retrieval Representation

The retrieval representation is the primary innovation proposed by SIMA.

A memory may have multiple retrieval representations.

For example:

```
Full Memory
├── Detailed Summary
├── Short Form
├── Retrieval Key
├── Entities
├── Topics
└── Embedding
```

Each representation has a different cost and purpose.

Representation	Primary Function
Full Memory	Final context
Detailed Summary	Deep ranking

Representation	Primary Function
Short Form	Semantic retrieval
Retrieval Key	Fast identification
Entities	Exact matching
Topics	Category routing
Embedding	Semantic similarity

6. Query Processing

A query enters the system.

Example:

```
"What was my idea about making AI memory retrieval more efficient?"
```

The query interpreter generates:

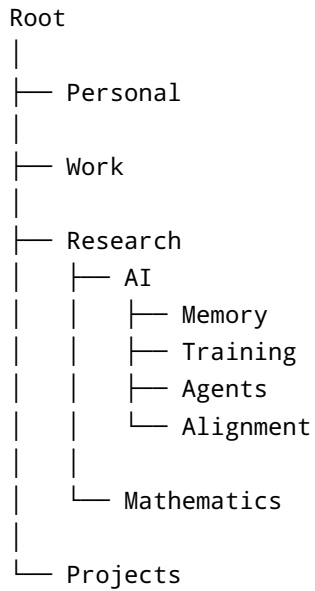
```
{
  "intent": "recall",
  "entities": [
    "AI",
    "memory",
    "retrieval",
    "efficiency"
  ],
  "categories": [
    "ai_research",
    "memory_architecture"
  ],
  "time_reference": null,
  "detail_requirement": "high"
}
```

The query is then passed to the category router.

7. Category Routing

The category router identifies likely memory partitions.

Conceptual structure:



A query is routed to the smallest likely search domain.

This is intended to reduce unnecessary retrieval operations.

8. Short-Form Search

Once routed, the system searches short-form representations.

Example:

```
MEM_001
"Category-indexed memory retrieval"

MEM_002
"Hierarchical memory routing"

MEM_003
"Graph-based associative retrieval"
```

```
MEM_004
"Long-context optimization"
```

The system generates candidate scores.

```
MEM_001 → 0.96
MEM_004 → 0.84
MEM_003 → 0.73
MEM_002 → 0.67
```

Only candidates above a defined threshold proceed.

9. Relevance Ranking

A conceptual relevance function:

```
R(m,q) =
  ws * S(m,q)
+ wc * C(m,q)
+ wt * T(m,q)
+ we * E(m,q)
+ wi * I(m)
```

Where:

```
S = semantic similarity
C = category compatibility
T = temporal relevance
E = entity overlap
I = importance
```

The weights are tunable.

An implementation should expose these parameters for benchmarking.

10. Memory Graph

SIMA optionally maintains a graph:

$G = (V, E)$

where:

V = memories
E = typed relationships

Relationship types:

related_to
derived_from
extends
supports
contradicts
supersedes
references

Example:

```
MEM_001
SIMA Architecture
|
├── extends ─── MEM_014
|
├── related_to ─── MEM_029
|
└── contradicts ─── MEM_031
```

The graph enables retrieval beyond nearest-neighbor similarity.

11. Multi-Hop Retrieval

Consider a query:

"What was the memory idea connected to my earlier graph theory work?"

The system may retrieve:

Query
↓
Memory Architecture
↓
Graph Theory
↓
Network Connectivity
↓
Retrieval Architecture

The final result is assembled from the relevant connected subgraph.

This is intended to address queries where the relevant answer is distributed across multiple memories.

12. Adaptive Expansion

Memory expansion is confidence-dependent.

Example:

Candidate Confidence = 0.98

Action:

Expand top 1-2 memories.

Example:

Candidate Confidence = 0.52

Action:

Expand additional candidates.
Perform graph traversal.
Re-rank.

Conceptually:

Expansion Depth = $f(\text{Confidence}, \text{Query Complexity})$

13. Context Assembly

The context assembler receives selected memories.

It should consider:

Relevance
Importance
Confidence
Recency
Relationship coherence
Token budget

Example:

[MEM_001]
Primary memory.

[MEM_014]
Supporting historical memory.

[MEM_029]
Related conceptual memory.

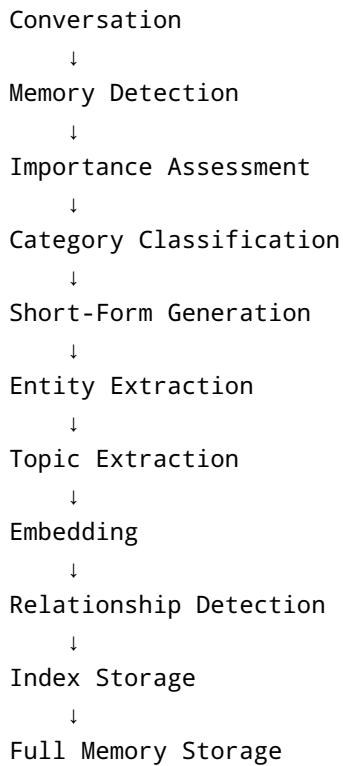
[MEM_031]
Conflicting memory requiring resolution.

The assembler may include provenance metadata so the model can distinguish:

Primary
Supporting
Historical
Conflicting
Uncertain

14. Memory Creation Pipeline

When new information is identified as memory-worthy:

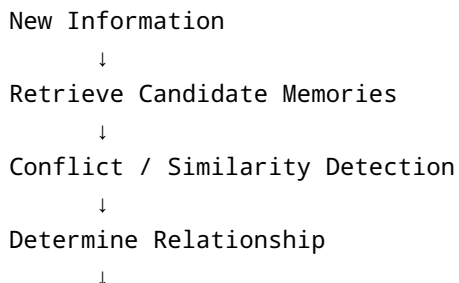


The system should preserve the original memory content rather than relying exclusively on compression.

The short form acts as an index.

15. Memory Update

When new information modifies an existing memory:



Update Memory Graph

↓

Update Short Form

↓

Update Full Memory

Possible actions:

append
update
supersede
merge
contradict
invalidate

This is necessary to prevent stale memories from dominating retrieval.

16. Context Budget Management

The context assembler should operate under a token budget:

B = Maximum Context Tokens Available

Each memory has an estimated expansion cost:

$\text{cost}(m)$

The system attempts to select a set M maximizing relevance:

maximize $\sum \text{relevance}(m)$

subject to:

$\sum \text{cost}(m) \leq B$

This creates a context-selection optimization problem.

17. Retrieval Cost Model

A conceptual retrieval cost is:

```
C_total =  
  C_query  
+ C_routing  
+ C_index  
+ C_ranking  
+ C_graph  
+ C_expansion  
+ C_context
```

SIMA's goal is to minimize:

```
C_total
```

while maintaining or improving:

```
Recall  
Precision  
Contextual Accuracy
```

The system should not assume that reducing tokens automatically improves performance.

The benchmark must measure the tradeoff.

18. Baseline Comparison

Every evaluation should compare:

```
Baseline A  
Flat Retrieval  
  
Baseline B  
Vector Search + Reranking  
  
Baseline C  
Hierarchical Retrieval
```

SIMA
Category + Short Form + Progressive Expansion

SIMA+
Category + Short Form + Graph + Adaptive Expansion

19. Benchmark Metrics

Retrieval

Recall@K
Precision@K
MRR
NDCG

Efficiency

Retrieved Tokens
Context Tokens
Latency
Compute Cost

Long-Term Memory

Single-Hop Recall
Multi-Hop Recall
Temporal Recall
Cross-Session Recall

Robustness

Stale Memory Resistance
Conflict Resolution
Duplicate Detection
False Retrieval Rate

20. Primary Research Hypothesis

The primary hypothesis is:

SIMA can maintain comparable or superior retrieval quality while reducing the amount of memory content that must be retrieved and injected into the active context.

Formally:

Maximize:

Recall Quality

Tokens + Compute + Latency

The hypothesis remains unvalidated until benchmarks are completed.

21. Scalability Model

The architecture is intended to support:

10^3 memories
 10^4 memories
 10^5 memories
 10^6 memories
 10^7+ memories

The expected advantage of hierarchical indexing should increase as the memory corpus grows.

However, this must be empirically tested.

The system should benchmark scaling behavior across corpus sizes.

22. Security and Integrity Considerations

Persistent AI memory introduces security risks.

SIMA should consider:

- Memory poisoning
- Unauthorized memory modification
- Retrieval injection
- Cross-user memory leakage
- Stale memory persistence
- Conflicting memory sources
- Malicious relationship creation

Memory objects should have provenance and access controls where applicable.

23. Open Research Questions

Representation

How short can a retrieval representation become before recall quality degrades?

Indexing

What category hierarchy produces optimal routing?

Retrieval

Should retrieval use symbolic, semantic, or hybrid search?

Graphs

Do explicit memory relationships outperform purely embedding-based retrieval?

Expansion

What confidence threshold should trigger full memory expansion?

Scaling

How does performance change from:

1,000
→ 100,000
→ 1,000,000
→ 100,000,000

memories?

Context

How much effective memory capacity can be achieved per token of active context?

24. Reference Implementation

The proposed reference implementation should expose the following interfaces:

```
memory = MemoryStore()

memory.create(
    category="ai_research",
    subcategory="memory_architecture",
    short_form="Category-indexed compressed memory retrieval.",
    full_content="..."
)

results = memory.retrieve(
    query="What was my memory architecture idea?"
)

context = memory.expand(
    results,
    token_budget=8000
)
```

A minimal API should remain model-agnostic.

25. Proposed Repository Structure

```
sima/
|
|— README.md
|— ARCHITECTURE.md
|— LICENSE
|— CONTRIBUTING.md
|
|— core/
|   |— memory_store.py
|   |— memory_index.py
|   |— category_router.py
```

```
|   |— relevance_ranker.py
|   |— memory_graph.py
|   |— graph_traversal.py
|   |— memory_expander.py
|   |— context_assembler.py
|
|— storage/
|   |— vector_store.py
|   |— document_store.py
|   |— graph_store.py
|
|— benchmarks/
|   |— datasets/
|   |— baselines/
|   |— sima/
|   |— evaluate.py
|
|— tests/
|
|— examples/
|   |— basic_retrieval.py
|   |— multi_hop_retrieval.py
```

26. Definition of Success

SIMA should be considered successful only if controlled experiments demonstrate measurable improvement over appropriate baselines.

A successful implementation would ideally demonstrate:

```
Same or better recall
+
Fewer retrieved tokens
+
Lower or comparable latency
+
Scalable memory corpus
```

The project should publish all benchmark methodology and results.

27. Final Architectural Thesis

SIMA proposes a fundamental separation:

Memory Storage
≠
Memory Retrieval
≠
Active Context

A memory can be:

Stored completely

while being:

Indexed compactly

and:

Retrieved selectively

and:

Expanded only when needed

The architecture therefore treats long-term memory as a persistent external information system rather than as a continuously active extension of the model's context.

The central research question is:

Can intelligent indexing and progressive expansion allow an AI system to operate over a memory corpus vastly larger than its active context window while preserving high-fidelity recall?

SIMA is an open-source attempt to answer that question experimentally.